VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

MICROPROCESSOR MICROCONTROLLER (CO3009)

LAB 2

TIMER INTERRUPT AND LED SCANNING

Class: L04 – Semester HK251
Submission Date: 17/10/2025

Instructor: MR. Ton Huynh Long

Student Name	Student ID
Phạm Công Võ	2313946

Ho Chi Minh City, September 2025

Table of Contents

List of Figures

1 Introduction

Timers are one of the most important features in modern micro-controllers. They allow us to measure how long something takes to execute, create non-blocking code, precisely control pin timing, and even run operating systems. In this manual, how to configure a timer using STM32CubeIDE is presented how to use them to flash an LED. Finally, students are proposed to finalize 10 exercises using timer interrupt for applications based LED Scanning.

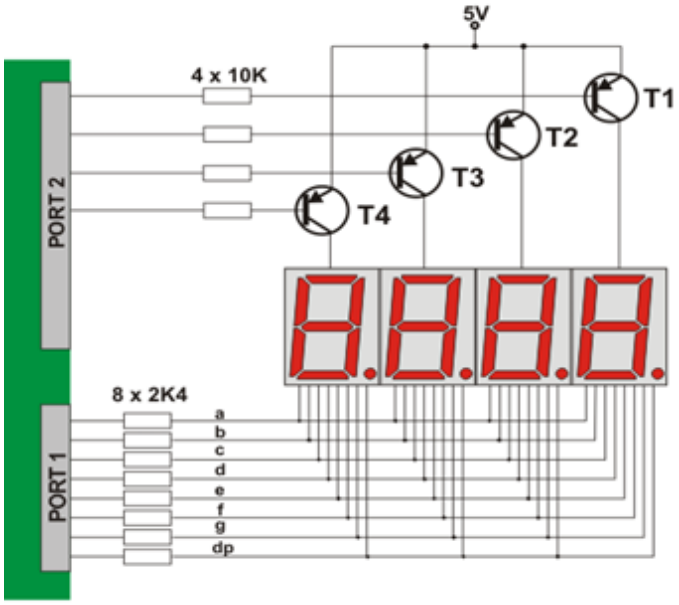

Figure 1: *Four seven segment LED interface for a micro-controller*

Design an interface for with multiple LED (seven segment or matrix) displays which is to be controlled is depends on the number of input and output pins needed for controlling all the LEDs in the given matrix display, the amount of current that each pin can source and sink and the speed at which the micro-controller can send out control signals. With all these specifications, interfacing can be done for 4 seven segment LEDs with a micro-controller is proposed in the figure above.

In the above diagram each seven segment display is having 8 internal LEDs, leading to the total number of LEDs is 32. However, not all the LEDs are required to turn ON, but one of them is needed. Therefore, only 12 lines are needed to control the whole 4 seven segment LEDs. By controlling with the micro-controller, we can turn ON an LED during a same interval T_S. Therfore, the period for controlling all 4 seven segment LEDs is $4T_S$. In other words, these LEDs are scanned at frequecy $f = 1/4T_S$. Finally, it is obviously that if the frequency is greater than 30Hz (e.g. f = 50Hz), it seems that all LEDs are turn ON at the same time.

In this manual, the timer interrupt is used to design the interval T_S for LED scanning. Unfortunately, the simulation on Proteus can not execute at high frequency, the frequency f is set to a low value (e.g. 1Hz). In a real implementation, this frequency should be 50Hz.

2 Timer Interrupt Setup

Step 1: Create a simple project, which LED connected to PA5. The manual can be found in the first lab.

Step 2: Check the clock source of the system on the tab **Clock Configuration** (from *.ioc file). In the default configuration, the internal clock source is used with 8MHz, as shown in the figure bellow.

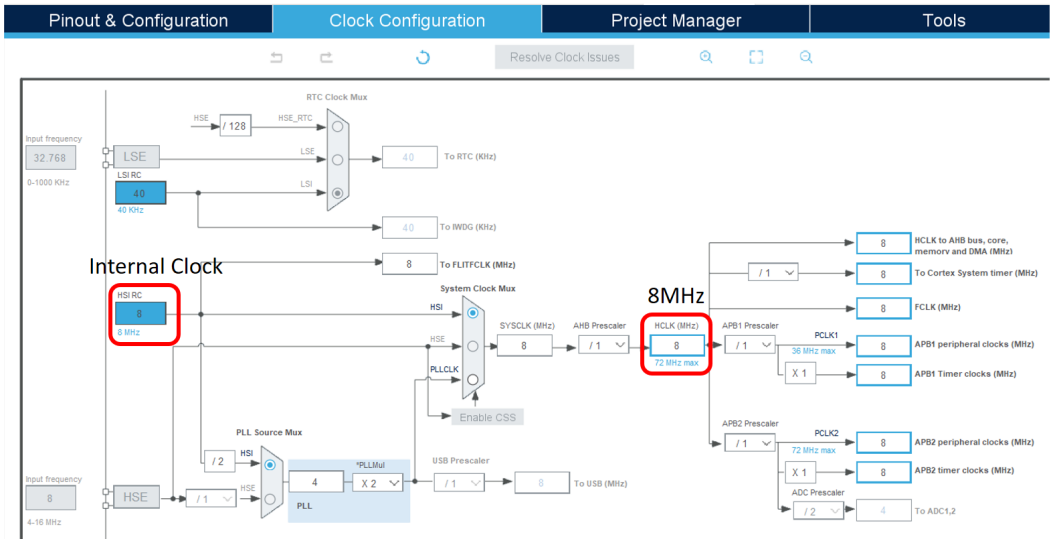

Figure 2: *Default clock source for the system*

Step 3: Configure the timer on the **Parameter Settings**, as follows:

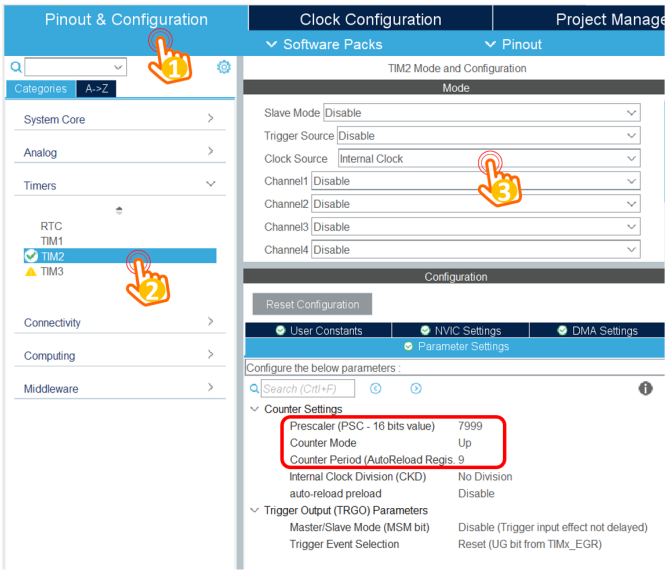

Figure 3: *Configure for Timer 2*

Select the clock source for timer 2 to the **Internal Clock**. Finally, set the prescaller and the counter to 7999 and 9, respectively. These values are explained as follows:

- The target is to set an interrupt timer to 10ms
- The clock source is 8MHz, by setting the prescaller to 7999, the input clock source to the timer is $\mathbf{8MHz/(7999+1) = 1000Hz}$.

- The interrupt is raised when the timer counter is counted from 0 to 9, meaning that the frequency is divided by 10, which is 100Hz.
- The frequency of the timer interrupt is 100Hz, meaning that the period is **1/100Hz = 10ms**.

Step 4: Enable the timer interrupt by switching to **NIVC Settings** tab, as follows:

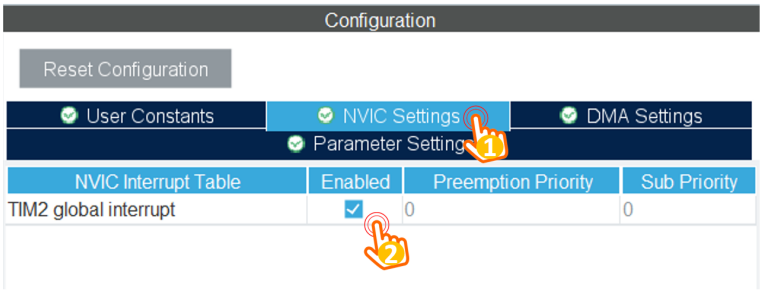

Figure 4: *Enable timer interrupt*

Finally, save the configuration file to generate the source code.

Step 5: On the **main()** function, call the timer init function, as follows:

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
  MX_TIM2_Init();

  /* USER CODE BEGIN 2 */
  HAL_TIM_Base_Start_IT(&htim2);
  /* USER CODE END 2 */g3

  while (1){

  }
}
```

Please put the init function in a right place to avoid conflicts when code generation is executed (e.g. ioc file is updated).

Step 6: Add the interrupt service routine function, this function is invoked every 10ms, as follows:

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){

}
/* USER CODE END 4 */
```

Step 7: To run a LED Blinky demo using interrupt, a short manual is presented as follows:

```
/* USER CODE BEGIN 4 */
int counter = 100;
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
  counter--;
  if(counter <= 0){
    counter = 100;
    HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
  }
}
/* USER CODE END 4 */
```

The **HAL_TIM_PeriodElapsedCallback** function is an infinite loop, which is invoked every cycle of the timer 2, in this case, is 10ms.

3 Exercise and Report

The source code and demo video for LAB2 are stored at: Link GitHub Repository – STM32 Lab 2.

3.1 Exercise 1

The first exercise show how to interface for multiple seven segment LEDs to STM32F103C6 micro-controller (MCU). Seven segment displays are common anode type, meaning that the anode of all LEDs are tied together as a single terminal and cathodes are left alone as individual pins.

In order to save the resource of the MCU, individual cathode pins from all the seven segment LEDs are connected together, and connect to 7 pins of the MCU. These pins are popular known as the **signal pins**. Meanwhile, the anode pin of each seven segment LEDs are controlled under a power enabling circuit, for instance, an PNP transistor. At a given time, only one seven segment LED is turned on. However, if the delay is small enough, it seems that all LEDs are enabling.

Implement the circuit simulation in Proteus with two 7-SEGMENT LEDs as following: Components

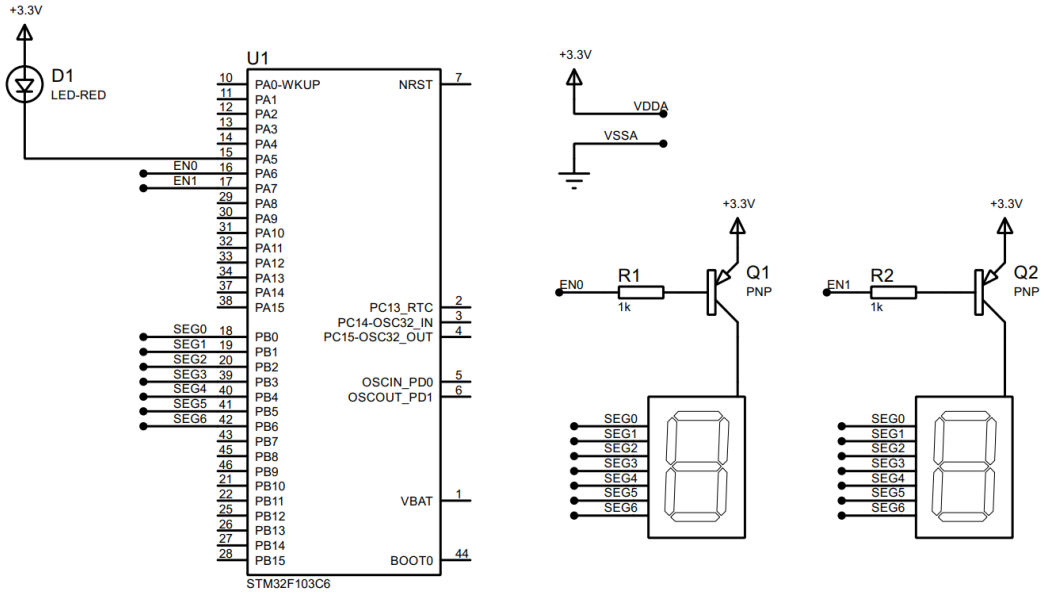

Figure 5: *Simulation schematic in Proteus*

used in the schematic are listed bellow:

- 7SEG-COM-ANODE (connected from PB0 to PB6)
- LED-RED
- PNP
- RES
- STM32F103C6

Students are proposed to use the function **display7SEG(int num)** in the Lab 1 in this exercise. Implement the source code in the interrupt callback function to display number **"1"** on the first seven segment and number **"2"** for second one. The switching time between 2 LEDs is half of second.

[Report Solution]:

Report 1: Capture your schematic from Proteus and show in the report.

The source code and Proteus simulation schematic are provided at STM32 Lab 2 : Two 7SEG Display.

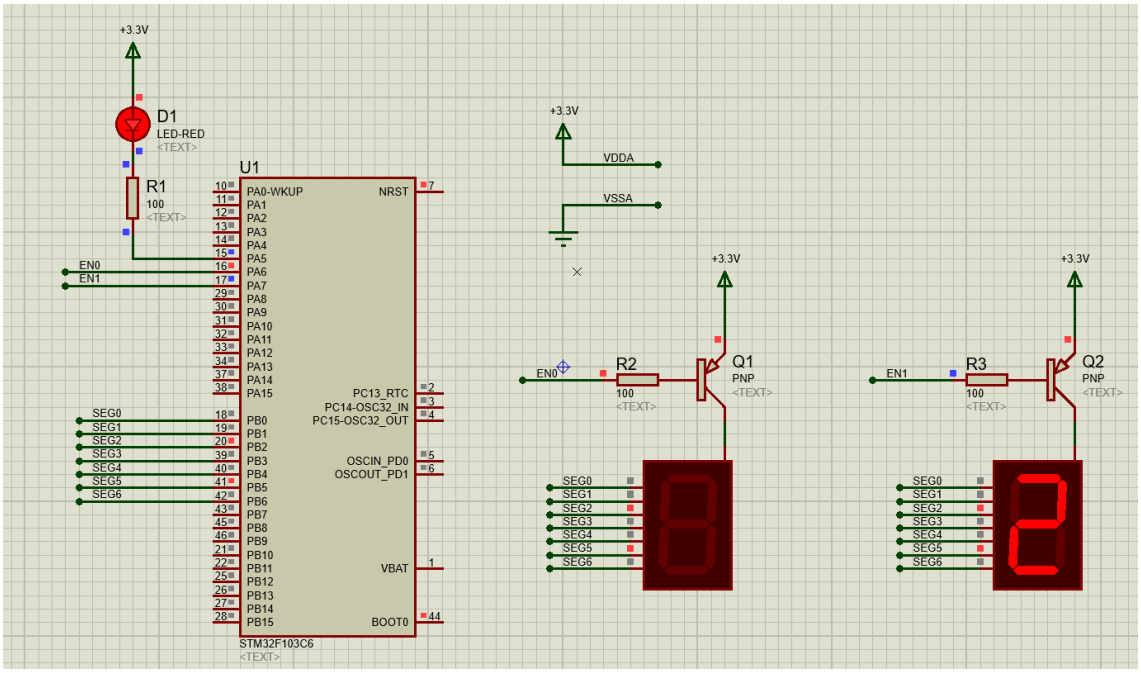

Figure 6: *Proteus Ex1 Two7SEG Display*

Report 2: Present your source code in the **HAL_TIM_PeriodElapsedCallback** function.

```
void display7SEG(int num)
{
  // Turn off all segments first
  HAL_GPIO_WritePin(GPIOB, SEG0_Pin | SEG1_Pin | SEG2_Pin | SEG3_Pin
    | SEG4_Pin | SEG5_Pin | SEG6_Pin, GPIO_PIN_SET);
  switch (num)
  {
  case 0: // Number 0: segments a b c d e f ON
    HAL_GPIO_WritePin(GPIOB, SEG0_Pin | SEG1_Pin | SEG2_Pin |
   SEG3_Pin | SEG4_Pin | SEG5_Pin, GPIO_PIN_RESET);
    break;
  case 1: // Number 1: segments b c ON
    HAL_GPIO_WritePin(GPIOB, SEG1_Pin | SEG2_Pin, GPIO_PIN_RESET);
    break;
  case 2: // Number 2: segments a b d e g ON
```

```
14      HAL_GPIO_WritePin(GPIOB, SEG0_Pin | SEG1_Pin | SEG3_Pin |
        SEG4_Pin | SEG6_Pin, GPIO_PIN_RESET);
15      break;
16    case 3: // Number 3: segments a b c d g ON
17      HAL_GPIO_WritePin(GPIOB, SEG0_Pin | SEG1_Pin | SEG2_Pin |
        SEG3_Pin | SEG6_Pin, GPIO_PIN_RESET);
18      break;
19    case 4: // Number 4: segments b c f g ON
20      HAL_GPIO_WritePin(GPIOB, SEG1_Pin | SEG2_Pin | SEG5_Pin |
        SEG6_Pin, GPIO_PIN_RESET);
21      break;
22    case 5: // Number 5: segments a c d f g ON
23      HAL_GPIO_WritePin(GPIOB, SEG0_Pin | SEG2_Pin | SEG3_Pin |
        SEG5_Pin | SEG6_Pin, GPIO_PIN_RESET);
24      break;
25    case 6: // Number 6: segments a c d e f g ON
26      HAL_GPIO_WritePin(GPIOB, SEG0_Pin | SEG2_Pin | SEG3_Pin |
        SEG4_Pin | SEG5_Pin | SEG6_Pin, GPIO_PIN_RESET);
27      break;
28    case 7: // Number 7: segments a b c ON
29      HAL_GPIO_WritePin(GPIOB, SEG0_Pin | SEG1_Pin | SEG2_Pin,
        GPIO_PIN_RESET);
30      break;
31    case 8: // Number 8: all segments ON
32      HAL_GPIO_WritePin(GPIOB, SEG0_Pin | SEG1_Pin | SEG2_Pin |
        SEG3_Pin | SEG4_Pin | SEG5_Pin | SEG6_Pin, GPIO_PIN_RESET);
33      break;
34    case 9: // Number 9: segments a b c d f g ON
35      HAL_GPIO_WritePin(GPIOB, SEG0_Pin | SEG1_Pin | SEG2_Pin |
        SEG3_Pin | SEG5_Pin | SEG6_Pin, GPIO_PIN_RESET);
36      break;
37    default: // Invalid input: show '-' using segment g
38      HAL_GPIO_WritePin(GPIOB, SEG6_Pin, GPIO_PIN_RESET);
39      break;
40    }
41  }
```

```
/*
 * - Enables one of the two 7-segment digits.
 * - Active-low control: ENx = 0 means enabled, ENx = 1 means
   disabled.
 */
void enableLED(int index)
{
  if (index == 0)
  {     // Enable digit 0, Disable digit 1
    HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_SET);
  }
  else if (index == 1)
  {     // Disable digit 0, Enable digit 1
    HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_RESET);
  }
}

volatile int currentLED = 0;

/*
 * HAL_TIM_PeriodElapsedCallback (triggered every 10 ms by TIM2).
 * - Updates software timers by calling timerRun().
 * - When timer 0 expires (every 500 ms), it toggles between two
   digits:
 *   Digit 0 displays number 1
 *   Digit 1 displays number 2
 * - A debug LED is toggled at the same time.
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2)
  {
    // Call software timer tick (executed every 10 ms)
    timerRun();
```

```
76      if (isTimerExpired(0))
77      {
78        // Reload timer 0 for another 500 ms (50 x 10 ms)
79        setTimer(0, 50);
80
81        if (currentLED == 0)
82        {
83          // Activate digit 0 and display number 1
84          enableLED(0);
85          display7SEG(1);
86          currentLED = 1;   // Switch to digit 1 next time
87        }
88        else
89        {
90          // Activate digit 1 and display number 2
91          enableLED(1);
92          display7SEG(2);
93          currentLED = 0;   // Switch back to digit 0 next time
94        }
95
96        // Toggle the debug LED for status indication
97        HAL_GPIO_TogglePin(GPIOA, LED_RED_Pin);
98      }
99    }
100 }
```

Short question: What is the frequency of the scanning process?

In this exercise, the program is designed to alternately display number "1" on the first seven-segment LED and number "2" on the second one. The switching time between two LEDs is given as **0.5 seconds**.

- The duration of one complete scanning cycle (LED1 → LED2 → LED1) is:

$$T = 0.5\,\text{s} + 0.5\,\text{s} = 1\,\text{s}$$

- The scanning frequency is therefore:

$$f = \frac{1}{T} = \frac{1}{1\,\text{s}} = 1\,\text{Hz}$$

The scanning frequency of the display process is **1 Hz**, meaning the two digits are refreshed once every second.

3.2 Exercise 2

Extend to 4 seven segment LEDs and two LEDs (connected to PA4, labeled as **DOT**) in the middle as following:

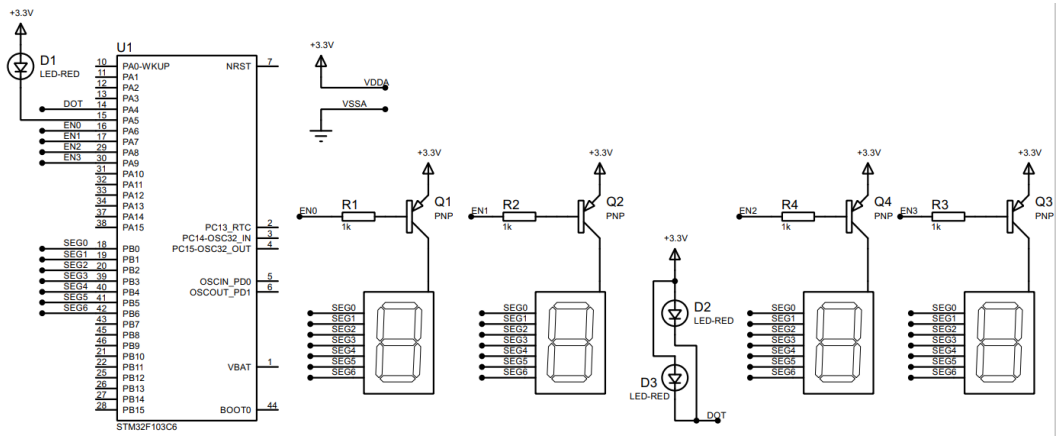

Figure 7: *Simulation schematic in Proteus*

Blink the two LEDs every second. Meanwhile, number 3 is displayed on the third seven segment and number 0 is displayed on the last one (to present 12 hour and a half). The switching time for each seven segment LED is also a half of second (500ms). **Implement your code in the timer interrupt function.**

[Report Solution]:

Report 1: Capture your schematic from Proteus and show in the report.
The source code and Proteus schematic are provided at STM32 Lab 2: Four7SEG DOTBlink.

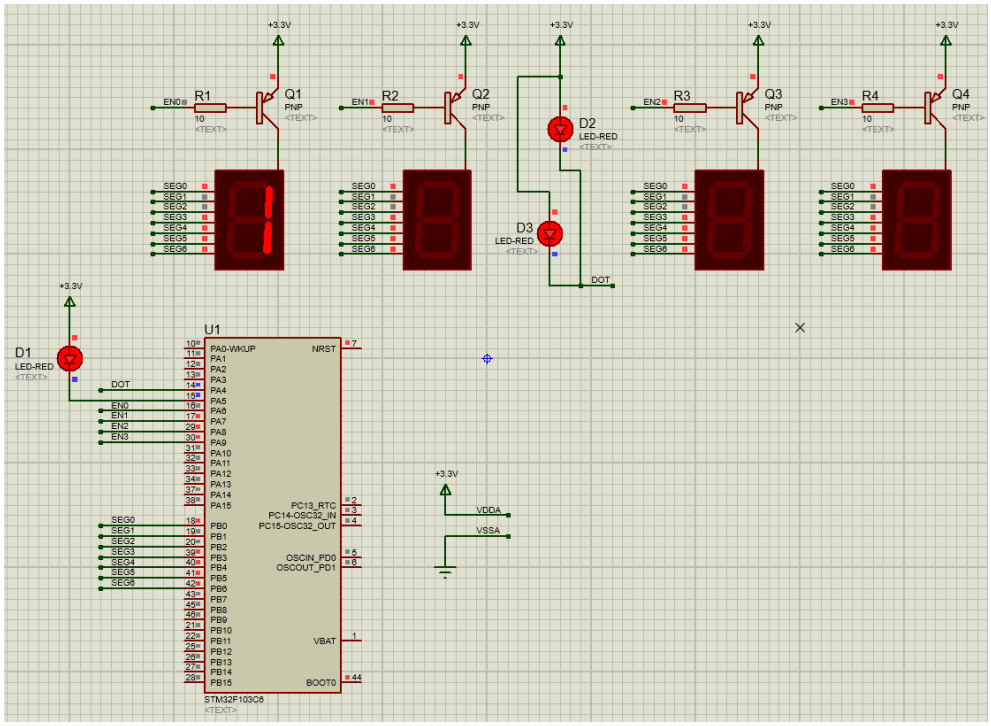

Figure 8: *Proteus Ex2 Four7SEG DOT Blink*

Report 2: Present your source code in the **HAL_TIM_PeriodElapsedCallback** function.

```
void enableLED(int index)
{       // Turn OFF all LEDs first
  HAL_GPIO_WritePin(GPIOA, EN0_Pin | EN1_Pin | EN2_Pin | EN3_Pin,
   GPIO_PIN_SET);
  switch (index)
  {
  case 0:   // Enable digit 0
    HAL_GPIO_WritePin(GPIOA, EN0_Pin, GPIO_PIN_RESET);
    break;
  case 1:   // Enable digit 1
    HAL_GPIO_WritePin(GPIOA, EN1_Pin, GPIO_PIN_RESET);
    break;
  case 2:   // Enable digit 2
    HAL_GPIO_WritePin(GPIOA, EN2_Pin, GPIO_PIN_RESET);
    break;
  case 3:   // Enable digit 3
    HAL_GPIO_WritePin(GPIOA, EN3_Pin, GPIO_PIN_RESET);
    break;
  default:
    break;
  }
}

int currentLED = 0;  // Current active digit index
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2)
  {     timerRun();       // Update software timers (tick = 10 ms)
    if (isTimerExpired(0))  // Scan 4 digits every 500 ms
    {
      setTimer(0, 50);
      switch (currentLED)
      {
      case 0: // Digit 0 shows "1"
          enableLED(0);
```

```
            display7SEG(1);
            break;
        case 1: // Digit 1 shows "2"
            enableLED(1);
            display7SEG(2);
            break;
        case 2: // Digit 2 shows "3"
            enableLED(2);
            display7SEG(3);
            break;
        case 3: // Digit 3 shows "0"
            enableLED(3);
            display7SEG(0);
            break;
        default:  // Error case, show "-"
            enableLED(currentLED);
            display7SEG(10);
            break;
        }
        currentLED++;
        if (currentLED >= 4) currentLED = 0; // Wrap back to digit 0
    }
    // Toggle DOT LED every 1 second
    if (isTimerExpired(1))
    {
        setTimer(1, 100);
        HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
    }
    // Toggle RED LED every 500 ms
    if (isTimerExpired(2))
    {
        setTimer(2, 50);
        HAL_GPIO_TogglePin(GPIOA, LED_RED_Pin);
    }
  }
}
```

Short question: What is the frequency of the scanning process?

The program is configured to switch between the four 7-segment digits every **0.5 seconds**. Therefore, one complete scanning cycle (digit 0 → digit 1 → digit 2 → digit 3 → digit 0) takes:

$$T = 4 \times 0.5\,\text{s} = 2\,\text{s}$$

The scanning frequency is calculated as:

$$f = \frac{1}{T} = \frac{1}{2\,\text{s}} = 0.5\,\text{Hz}$$

The scanning frequency of the 4-digit display is **0.5 Hz**, meaning the system refreshes the entire set of digits once every 2 seconds.

3.3 Exercise 3

Implement a function named **update7SEG(int index)**. An array of 4 integer numbers are declared in this case. The code skeleton in this exercise is presented as following:

```
const int MAX_LED = 4;
int index_led = 0;
int led_buffer[4] = {1, 2, 3, 4};
void update7SEG(int index){
    switch (index){
        case 0:
            //Display the first 7SEG with led_buffer[0]
            break;
        case 1:
            //Display the second 7SEG with led_buffer[1]
            break;
        case 2:
            //Display the third 7SEG with led_buffer[2]
            break;
        case 3:
            //Display the forth 7SEG with led_buffer[3]
            break;
        default:
            break;
    }
}
```

This function should be invoked in the timer interrupt, e.g update7SEG(index_led++). The variable **index_led** is updated to stay in a valid range, which is from 0 to 3.

[Report Solution]:

Report 1: Present the source code of the update7SEG function.

The source code and Proteus simulation schematic are provided at STM32 Lab 2: Update7SEG.

```
const int MAX_LED = 4;                // Number of 7-segment LEDs
int index_led = 0;                    // Current scanning index
int led_buffer[4] = {1, 2, 3, 4};     // Values for each LED

// Update one LED with value from buffer
void update7SEG(int index)
{
  switch (index)
  {
  case 0:
      enableLED(0);
      display7SEG(led_buffer[0]);
      break;
  case 1:
      enableLED(1);
      display7SEG(led_buffer[1]);
      break;
  case 2:
      enableLED(2);
      display7SEG(led_buffer[2]);
      break;
  case 3:
      enableLED(3);
      display7SEG(led_buffer[3]);
      break;
  default:
      HAL_GPIO_WritePin(GPIOA, EN0_Pin|EN1_Pin|EN2_Pin|EN3_Pin,
   GPIO_PIN_SET);
      break;
  }
}

```

```
void scanLED()
{
  update7SEG(index_led);  // Update current LED
  index_led++;
  if (index_led >= MAX_LED) index_led = 0; // Wrap back
}
```

Report 2: Present the source code in the HAL_TIM_PeriodElapsedCallback.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2)
  {
    timerRun(); // Update software timers

    // Scan 7-segment LEDs every 500 ms (EX3)
    if (isTimerExpired(0))
    {
      setTimer(0, 50);
      scanLED();
    }

    // Toggle DOT LED every 1s
    if (isTimerExpired(1))
    {
      setTimer(1, 100);
      HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
    }
    // Toggle RED LED every 500 ms
    if (isTimerExpired(2))
    {
      setTimer(2, 50);
      HAL_GPIO_TogglePin(GPIOA, LED_RED_Pin);
    }
  }
}
```

Students are proposed to change the values in the **led_buffer** array for unit test this function, which is used afterward.

3.4 Exercise 4

Change the period of invoking update7SEG function in order to set the frequency of 4 seven segment LEDs to 1Hz. The DOT is still blinking every second.

[Report Solution]:

Report 1: Present the source code in the **HAL_TIM_PeriodElapsedCallback**.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  if (htim->Instance == TIM2)
  {
    // Run software timers every 10ms
    timerRun();

    // Scan 7-segment LEDs every 250ms (EX4) - 1s for 4 LED
    if (isTimerExpired(0))
    {
      setTimer(0, 25);
      scanLED();   // Update current 7-segment LED
    }

    // Toggle dot LED every 1s
    if (isTimerExpired(1))
    {
      setTimer(1, 100);
      HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
    }

    // Toggle red LED every 500ms
    if (isTimerExpired(2))
    {
      setTimer(2, 50);
      HAL_GPIO_TogglePin(GPIOA, LED_RED_Pin);
    }
  }
}
```

3.5 Exercise 5

Implement a digital clock with **hour** and **minute** information displayed by 2 seven segment LEDs. The code skeleton in the **main** function is presented as follows:

```
int hour = 15, minute = 8, second = 50;

while(1){
    second++;
    if (second >= 60){
        second = 0;
        minute++;
    }
    if(minute >= 60){
        minute = 0;
        hour++;
    }
    if(hour >=24){
        hour = 0;
    }
    updateClockBuffer();
    HAL_Delay(1000);
}
```

The function **updateClockBuffer** will generate values for the array **led_buffer** according to the values of hour and minute. In the case these values are 1 digit number, digit 0 is added.

[Report Solution]:

Report 1: Present the source code in the **updateClockBuffer** function.

The source code and Proteus simulation schematic are provided at STM32 Lab 2: DigitalClock.

```
// Update the 7-segment buffer with the current time
void updateClockBuffer(void) {
    led_buffer[0] = hour / 10;   // Tens digit of hour (1 in 15)
    led_buffer[1] = hour % 10;   // Units digit of hour (5 in 15)
    led_buffer[2] = minute / 10; // Tens digit of minute (0 in 08)
    led_buffer[3] = minute % 10; // Units digit of minute (8 in 08)
    // Example result: display "1508" on 4-digit 7-segment display
}
```

3.6 Exercise 6

The main target from this exercise to reduce the complexity (or reduce code processing) in the timer interrupt. The time consumed in the interrupt can lead to the nested interrupt issue, which can crash the whole system. A simple solution can disable the timer whenever the interrupt occurs, the enable it again. However, the real-time processing is not guaranteed anymore.

In this exercise, a software timer is created and its counter is count down every timer interrupt is raised (every 10ms). By using this timer, the **Hal_Delay(1000)** in the main function is removed. In a MCU system, non-blocking delay is better than blocking delay. The details to create a software timer are presented bellow. The source code is added to your current program, **do not delete the source code you have on Exercise 5.**

Step 1: Declare variables and functions for a software timer, as following:

```
/* USER CODE BEGIN 0 */
int timer0_counter = 0;
int timer0_flag = 0;
int TIMER_CYCLE = 10;
void setTimer0(int duration){
  timer0_counter = duration /TIMER_CYCLE;
  timer0_flag = 0;
}
void timer_run(){
  if(timer0_counter > 0){
    timer0_counter--;
    if(timer0_counter == 0) timer0_flag = 1;
  }
}
/* USER CODE END 0 */
```

Please change the **TIMER_CYCLE** to your timer interrupt period. In the manual code above, it is **10ms**.

Step 2: The **timer_run()** is invoked in the timer interrupt as following:

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){

  timer_run();

  //YOUR OTHER CODE
}
```

Step 3: Use the timer in the main function by invoked setTimer0 function, then check for its flag (timer0_flag). An example to blink an LED connected to PA5 using software timer is shown as follows:

```
1 setTimer0(1000);
2 while (1){
3     if(timer0_flag == 1){
4         HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);
5         setTimer0(2000);
6     }
7 }
```

[Report Solution]:

In microcontroller systems, to avoid using blocking delay functions (e.g., `HAL_Delay`), a software timer is implemented based on periodic interrupts. The mechanism works as follows:

1. The hardware timer is configured to generate an interrupt every texttttTIMER_CYCLE milliseconds (e.g., 10 ms).
2. When `setTimer0(duration)` is called, the variable `timer0_counter` is assigned the value `(duration / TIMER_CYCLE)`, and the flag `timer0_flag` is reset.
3. At each timer interrupt, the function `timer_run()` decreases `timer0_counter`. When the counter reaches zero, `timer0_flag` is set to 1 to signal that the time event has occurred.
4. In the main loop, the program checks `timer0_flag`. When this flag is set, the required action (e.g., toggling an LED) is executed, and the timer can be reinitialized if needed.

By using this mechanism, the main program is no longer blocked by **HAL_Delay**, resulting in non-blocking execution and improved real-time responsiveness.

Report 1: Omitting `setTimer0(1000)` - line 1 of the code above is miss.

If the timer is not initialized by `setTimer0(1000)`, the variable `timer0_counter` keeps its default value of 0. In this case, inside the timer interrupt, the condition `if (timer0_counter > 0)` will never be true, so `timer0_flag` will never be set to 1. As a result, in the main loop, the condition `if (timer0_flag == 1)` is never satisfied, and the LED will not blink.

Reason: No counter is initialized, so the software timer never generates an event.

Report 2: Replacing `setTimer0(1000);` with `setTimer0(1);`

When `setTimer0(1);` is called, the value of `timer0_counter` is calculated as $1/10 = 0$ (due to integer division in C). Thus, the counter immediately becomes 0, and `timer0_flag` is never set. In subsequent calls to `timer_run()`, the counter remains 0, and no event is generated. Consequently, the LED never changes its state.

Reason: The configured duration (1 ms) is smaller than the interrupt cycle (10 ms). Integer division truncates the result to 0, so the timer never works.

Report 3: Replacing `setTimer0(1000);` with `setTimer0(10);`

In this case, `timer0_counter` is assigned 10/10 = 1. After one timer interrupt (i.e., 10 ms), `timer_run()` decreases the counter to 0 and sets `timer0_flag = 1`. At this moment, the main loop detects the event and toggles the LED.

Result: The LED will blink every 10 ms.

Difference: Unlike Report 1 and Report 2, here the counter is initialized with a valid value (1), so the event is properly generated.

Comparison of the Three Cases:

Case	Duration input	Counter value	Event (flag)	LED status
Report 1 (no `set`)	—	0 (default)	Never	LED unchanged
Report 2 (`setTimer0(1)`)	1 ms	0	Never	LED unchanged
Report 3 (`setTimer0(10)`)	10 ms	1	After 10 ms	LED blinks

Conclusion and Recommendation:

- Initializing the timer with `setTimer0()` is mandatory for the software timer to operate.
- If the duration parameter is smaller than the interrupt cycle (`TIMER_CYCLE`), integer division will yield 0, preventing the timer from generating events.
- When the parameter is an exact multiple of the cycle (e.g., `setTimer0(10)` with `TIMER_CYCLE = 10`), the counter is valid, and the event is triggered at the expected time.

This solution enables a non-blocking delay mechanism, ensures real-time performance, and reduces the risk of nested interrupts in embedded systems

3.7 Exercise 7

Upgrade the source code in Exercise 5 (update values for hour, minute and second) by using the software timer and remove the HAL_Delay function at the end. Moreover, the DOT (connected to PA4) of the digital clock is also moved to main function.

[Report Solution]:

Report 1: Present your source code in the while loop on main function.

The source code and Proteus schematic are provided at STM32 Lab 2: DigitalClock NoDelay.

```
HAL_TIM_Base_Start_IT(&htim2);
while (1)
{   // Timer 1: toggle DOT LED every 0.5 second
    if (isTimerExpired(1))
    {
        setTimer(1, 50);          // 50 * 10ms = 500ms = 0.5s
```

```
        HAL_GPIO_TogglePin(GPIOA, DOT_Pin); // Toggle DOT LED
    }
    // Timer 3: update clock every 1 second
    if (isTimerExpired(3))
    {
        // Increase second and handle overflow
        second++;
        if (second >= 60) {  // If second reaches 60
            second = 0;      // Reset second
            minute++;        // Increase minute
        }
        if (minute >= 60) {  // If minute reaches 60
            minute = 0;      // Reset minute
            hour++;          // Increase hour
        }
        if (hour >= 24) {    // If hour reaches 24
            hour = 0;        // Reset hour to 0 (new day)
        }
        updateClockBuffer(); // Update 7 segment buffer
        setTimer(3, 100);    // 100 * 10ms = 1s
    }
}
```

3.8 Exercise 8

Move also the update7SEG() function from the interrupt timer to the main. Finally, the timer interrupt only used to handle software timers. All processing (or complex computations) is move to an infinite loop on the main function, optimizing the complexity of the interrupt handler function.

[Report Solution]:

Report 1: Present your source code in the the main function. In the case more extra functions are used (e.g. the second software timer), present them in the report as well.

The source code and Proteus simulation schematic are provided at STM32 Lab 2: DigitalClock Final.

```
// Enable timer interrupt after initialization
HAL_TIM_Base_Start_IT(&htim2);
while (1)
{    // Timer 0: scan 7 segment LEDs every 250 ms for smooth display
```

```
    if (isTimerExpired(0))
    {
        setTimer(0, 25);        // 25 * 10 ms = 250 ms
        scanLED();              // Scan through all 7 segment LEDs
    }
    // Timer 1: toggle DOT LED every 0.5 second
    if (isTimerExpired(1))
    {
        setTimer(1, 50);        // 50 * 10 ms = 500 ms
        HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
    }
    // Timer 2: toggle RED LED every 0.25 second
    if (isTimerExpired(2))
    {
        setTimer(2, 25);        // 25 * 10 ms = 250 ms
        HAL_GPIO_TogglePin(GPIOA, LED_RED_Pin);
    }
    // Timer 3: update clock every 1 second
    if (isTimerExpired(3))
    {   // Increase time and handle overflow
        second++;
        if (second >= 60) {
            second = 0;
            minute++;
        }
        if (minute >= 60) {
            minute = 0;
            hour++;
        }
        if (hour >= 24) {
            hour = 0;   // Reset hour when a new day starts
        }
        updateClockBuffer();    // Update 7 segment buffer
        setTimer(3, 100);       // 100 * 10 ms = 1 second
    }
}
```

3.9 Exercise 9

This is an extra works for this lab. A LED Matrix is added to the system. A reference design is shown in figure bellow:

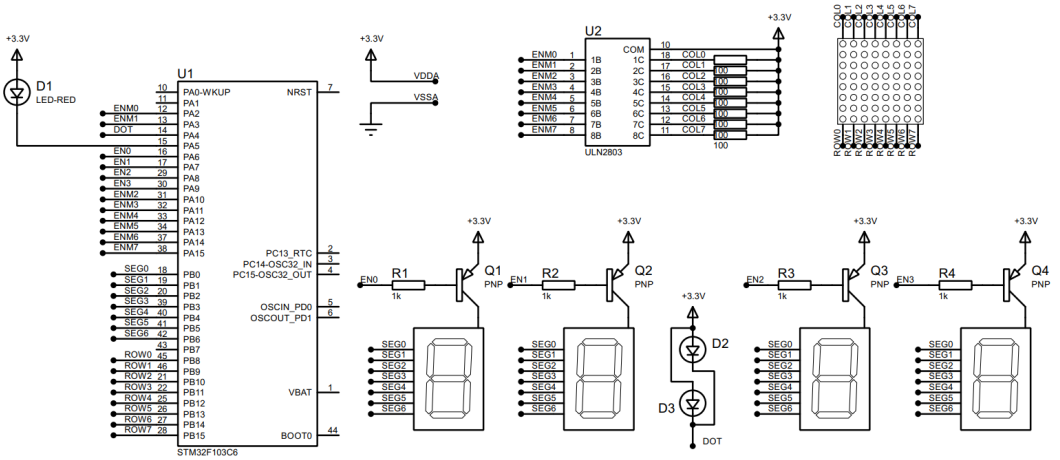

Figure 9: *LED matrix is added to the simulation*

In this schematic, two new components are added, including the **MATRIX-8X8-RED** and **ULN2803**, which is an NPN transistor array to enable the power supply for a column of the LED matrix. Students can change the enable signal (from ENM0 to ENM7) if needed. Finally, the data signal (from ROW0 to ROW7) is connected to PB8 to PB15.

Report 1: Present the schematic of your system by capturing the screen in Proteus.

Report 2: Implement the function, updateLEDMatrix(int index), which is similarly to 4 seven led segments.

```
const int MAX_LED_MATRIX = 8;
int index_led_matrix = 0;
uint8_t matrix_buffer[8] = {0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07
    , 0x08};
void updateLEDMatrix(int index){
    switch (index){
        case 0:
            break;
        case 1:
            break;
        case 2:
            break;
        case 3:
            break;
        case 4:
```

```
            break;
        case 5:
            break;
        case 6:
            break;
        case 7:
            break;
        default:
            break;
    }
}
```

Student are free to choose the invoking frequency of this function. However, this function is supposed to invoked in main function. Finally, please update the **matrix_buffer** to display character **"A"**.

[Report Solution]:

Report 1: Present the schematic of your system by capturing the screen in Proteus.

The source code and Proteus schematic are provided at STM32 Lab 2: LEDMatrix DisplayA.

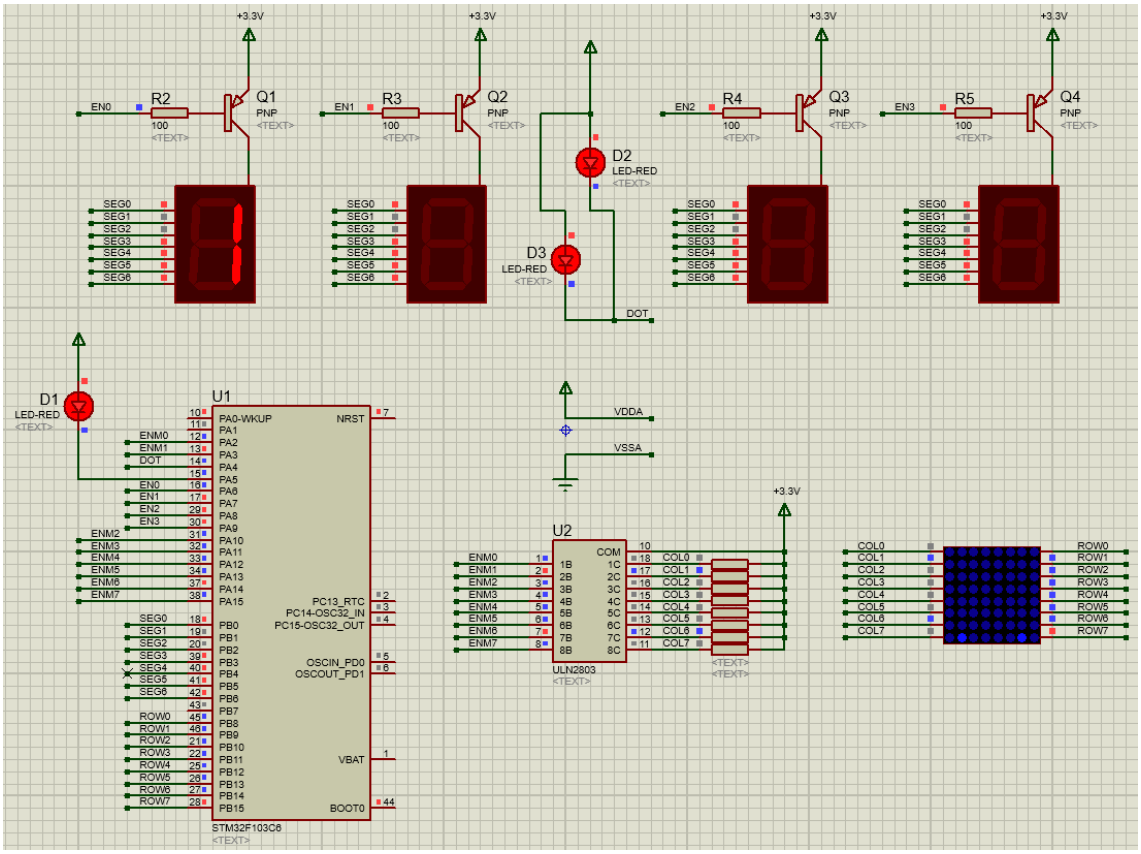

Figure 10: *Proteus EX9 LED Matrix DisplayA*

Report 2: Implement the function, updateLEDMatrix(int index), which is similarly to 4 seven led segments.

```
/* ===== PART 1: GLOBAL VARIABLES FOR 7-SEGMENT CLOCK ===== */
const int MAX_LED = 4;                // Four 7-segment LEDs
int index_led = 0;                    // Current active LED
int hour = 15, minute = 8, second = 50; // Initial time 15:08:50
int led_buffer[4] = {0, 0, 0, 0};

/* ===== PART 2: GLOBAL VARIABLES FOR 8x8 LED MATRIX ===== */
const int MAX_LED_MATRIX = 8; // Matrix has 8 rows
int index_led_matrix = 0;     // Current active row (0 to 7)

// Pattern to display letter "A" on LED Matrix 8x8
// Each byte represents one row, each bit represents one LED
uint8_t matrix_buffer[8] = {
    0x18, // 00011000 - Top of A
    0x24, // 00100100 - Upper diagonal
    0x42, // 01000010 - Side parts of A
    0x42, // 01000010 - Side parts of A
    0x7E, // 01111110 - Middle horizontal bar
    0x42, // 01000010 - Side parts of A
    0x42, // 01000010 - Side parts of A
    0x42  // 01000010 - Bottom of A
};

/* ===== PART 3: FUNCTIONS FOR LED MATRIX CONTROL ===== */
GPIO_PinState getBitState(uint8_t hexa, int index)
{   // Temporary array to hold bits
  int arr[8] = {0, 0, 0, 0, 0, 0, 0, 0};
  // Convert hex number into bits
  for (int i = 7; i >= 0; --i)
  {
    int mod = hexa % 2;  // Get last bit
    hexa = hexa / 2;     // Shift right
    arr[i] = mod;        // Save bit
  }
```

```
35   // Return bit state
36   if (arr[index] == 1)
37     return GPIO_PIN_SET;    // Bit = 1, LED ON
38   return GPIO_PIN_RESET;    // Bit = 0, LED OFF
39 }
40 
41 // Update column data for selected row
42 void updateLEDMatrix(int index)
43 {
44   switch (index)
45   {
46   case 0: // Row 0
47     HAL_GPIO_WritePin(GPIOA, ENM0_Pin, getBitState(matrix_buffer[0],
       0));
48     HAL_GPIO_WritePin(GPIOA, ENM1_Pin, getBitState(matrix_buffer[0],
       1));
49     HAL_GPIO_WritePin(GPIOA, ENM2_Pin, getBitState(matrix_buffer[0],
       2));
50     HAL_GPIO_WritePin(GPIOA, ENM3_Pin, getBitState(matrix_buffer[0],
       3));
51     HAL_GPIO_WritePin(GPIOA, ENM4_Pin, getBitState(matrix_buffer[0],
       4));
52     HAL_GPIO_WritePin(GPIOA, ENM5_Pin, getBitState(matrix_buffer[0],
       5));
53     HAL_GPIO_WritePin(GPIOA, ENM6_Pin, getBitState(matrix_buffer[0],
       6));
54     HAL_GPIO_WritePin(GPIOA, ENM7_Pin, getBitState(matrix_buffer[0],
       7));
55     break;
56 
57   case 1: // Row 1
58     HAL_GPIO_WritePin(GPIOA, ENM0_Pin, getBitState(matrix_buffer[1],
       0));
59     HAL_GPIO_WritePin(GPIOA, ENM1_Pin, getBitState(matrix_buffer[1],
       1));
60     HAL_GPIO_WritePin(GPIOA, ENM2_Pin, getBitState(matrix_buffer[1],
```

```
    2));
61    HAL_GPIO_WritePin(GPIOA, ENM3_Pin, getBitState(matrix_buffer[1],
      3));
62    HAL_GPIO_WritePin(GPIOA, ENM4_Pin, getBitState(matrix_buffer[1],
      4));
63    HAL_GPIO_WritePin(GPIOA, ENM5_Pin, getBitState(matrix_buffer[1],
      5));
64    HAL_GPIO_WritePin(GPIOA, ENM6_Pin, getBitState(matrix_buffer[1],
      6));
65    HAL_GPIO_WritePin(GPIOA, ENM7_Pin, getBitState(matrix_buffer[1],
      7));
66    break;
67
68  case 2: // Row 2
69    HAL_GPIO_WritePin(GPIOA, ENM0_Pin, getBitState(matrix_buffer[2],
      0));
70    HAL_GPIO_WritePin(GPIOA, ENM1_Pin, getBitState(matrix_buffer[2],
      1));
71    HAL_GPIO_WritePin(GPIOA, ENM2_Pin, getBitState(matrix_buffer[2],
      2));
72    HAL_GPIO_WritePin(GPIOA, ENM3_Pin, getBitState(matrix_buffer[2],
      3));
73    HAL_GPIO_WritePin(GPIOA, ENM4_Pin, getBitState(matrix_buffer[2],
      4));
74    HAL_GPIO_WritePin(GPIOA, ENM5_Pin, getBitState(matrix_buffer[2],
      5));
75    HAL_GPIO_WritePin(GPIOA, ENM6_Pin, getBitState(matrix_buffer[2],
      6));
76    HAL_GPIO_WritePin(GPIOA, ENM7_Pin, getBitState(matrix_buffer[2],
      7));
77    break;
78
79  case 3: // Row 3
80    HAL_GPIO_WritePin(GPIOA, ENM0_Pin, getBitState(matrix_buffer[3],
      0));
81    HAL_GPIO_WritePin(GPIOA, ENM1_Pin, getBitState(matrix_buffer[3],
```

```
    1));
82    HAL_GPIO_WritePin(GPIOA, ENM2_Pin, getBitState(matrix_buffer[3],
      2));
83    HAL_GPIO_WritePin(GPIOA, ENM3_Pin, getBitState(matrix_buffer[3],
      3));
84    HAL_GPIO_WritePin(GPIOA, ENM4_Pin, getBitState(matrix_buffer[3],
      4));
85    HAL_GPIO_WritePin(GPIOA, ENM5_Pin, getBitState(matrix_buffer[3],
      5));
86    HAL_GPIO_WritePin(GPIOA, ENM6_Pin, getBitState(matrix_buffer[3],
      6));
87    HAL_GPIO_WritePin(GPIOA, ENM7_Pin, getBitState(matrix_buffer[3],
      7));
88    break;
89
90  case 4: // Row 4
91    HAL_GPIO_WritePin(GPIOA, ENM0_Pin, getBitState(matrix_buffer[4],
      0));
92    HAL_GPIO_WritePin(GPIOA, ENM1_Pin, getBitState(matrix_buffer[4],
      1));
93    HAL_GPIO_WritePin(GPIOA, ENM2_Pin, getBitState(matrix_buffer[4],
      2));
94    HAL_GPIO_WritePin(GPIOA, ENM3_Pin, getBitState(matrix_buffer[4],
      3));
95    HAL_GPIO_WritePin(GPIOA, ENM4_Pin, getBitState(matrix_buffer[4],
      4));
96    HAL_GPIO_WritePin(GPIOA, ENM5_Pin, getBitState(matrix_buffer[4],
      5));
97    HAL_GPIO_WritePin(GPIOA, ENM6_Pin, getBitState(matrix_buffer[4],
      6));
98    HAL_GPIO_WritePin(GPIOA, ENM7_Pin, getBitState(matrix_buffer[4],
      7));
99    break;
100 case 5: // Row 5
101   HAL_GPIO_WritePin(GPIOA, ENM0_Pin, getBitState(matrix_buffer[5],
      0));
```

```
102     HAL_GPIO_WritePin(GPIOA, ENM0_Pin, getBitState(matrix_buffer[5],
        0));
103     HAL_GPIO_WritePin(GPIOA, ENM1_Pin, getBitState(matrix_buffer[5],
        1));
104     HAL_GPIO_WritePin(GPIOA, ENM2_Pin, getBitState(matrix_buffer[5],
        2));
105     HAL_GPIO_WritePin(GPIOA, ENM3_Pin, getBitState(matrix_buffer[5],
        3));
106     HAL_GPIO_WritePin(GPIOA, ENM4_Pin, getBitState(matrix_buffer[5],
        4));
107     HAL_GPIO_WritePin(GPIOA, ENM5_Pin, getBitState(matrix_buffer[5],
        5));
108     HAL_GPIO_WritePin(GPIOA, ENM6_Pin, getBitState(matrix_buffer[5],
        6));
109     HAL_GPIO_WritePin(GPIOA, ENM7_Pin, getBitState(matrix_buffer[5],
        7));
110     break;
111   case 6: // Row 6
112     HAL_GPIO_WritePin(GPIOA, ENM0_Pin, getBitState(matrix_buffer[6],
        0));
113     HAL_GPIO_WritePin(GPIOA, ENM1_Pin, getBitState(matrix_buffer[6],
        1));
114     HAL_GPIO_WritePin(GPIOA, ENM2_Pin, getBitState(matrix_buffer[6],
        2));
115     HAL_GPIO_WritePin(GPIOA, ENM3_Pin, getBitState(matrix_buffer[6],
        3));
116     HAL_GPIO_WritePin(GPIOA, ENM4_Pin, getBitState(matrix_buffer[6],
        4));
117     HAL_GPIO_WritePin(GPIOA, ENM5_Pin, getBitState(matrix_buffer[6],
        5));
118     HAL_GPIO_WritePin(GPIOA, ENM6_Pin, getBitState(matrix_buffer[6],
        6));
119     HAL_GPIO_WritePin(GPIOA, ENM7_Pin, getBitState(matrix_buffer[6],
        7));
120     break;
121   case 7: // Row 7
```

```
122     HAL_GPIO_WritePin(GPIOA, ENM0_Pin, getBitState(matrix_buffer[7],
        0));
123     HAL_GPIO_WritePin(GPIOA, ENM1_Pin, getBitState(matrix_buffer[7],
        1));
124     HAL_GPIO_WritePin(GPIOA, ENM2_Pin, getBitState(matrix_buffer[7],
        2));
125     HAL_GPIO_WritePin(GPIOA, ENM3_Pin, getBitState(matrix_buffer[7],
        3));
126     HAL_GPIO_WritePin(GPIOA, ENM4_Pin, getBitState(matrix_buffer[7],
        4));
127     HAL_GPIO_WritePin(GPIOA, ENM5_Pin, getBitState(matrix_buffer[7],
        5));
128     HAL_GPIO_WritePin(GPIOA, ENM6_Pin, getBitState(matrix_buffer[7],
        6));
129     HAL_GPIO_WritePin(GPIOA, ENM7_Pin, getBitState(matrix_buffer[7],
        7));
130     break;
131   default:
132     break;
133   }
134 }
135 
136 // Scan LED matrix rows
137 void scanLEDMatrix()
138 {
139   switch (index_led_matrix)
140   {
141   case 0: // Enable row 0
142     HAL_GPIO_WritePin(GPIOB, ROW7_Pin, GPIO_PIN_RESET);
143     HAL_GPIO_WritePin(GPIOB, ROW0_Pin, GPIO_PIN_SET);
144     break;
145   case 1: // Enable row 1
146     HAL_GPIO_WritePin(GPIOB, ROW0_Pin, GPIO_PIN_RESET);
147     HAL_GPIO_WritePin(GPIOB, ROW1_Pin, GPIO_PIN_SET);
148     break;
149   case 2: // Enable row 2
```

```
150     HAL_GPIO_WritePin(GPIOB, ROW1_Pin, GPIO_PIN_RESET);
151     HAL_GPIO_WritePin(GPIOB, ROW2_Pin, GPIO_PIN_SET);
152     break;
153   case 3: // Enable row 3
154     HAL_GPIO_WritePin(GPIOB, ROW2_Pin, GPIO_PIN_RESET);
155     HAL_GPIO_WritePin(GPIOB, ROW3_Pin, GPIO_PIN_SET);
156     break;
157   case 4: // Enable row 4
158     HAL_GPIO_WritePin(GPIOB, ROW3_Pin, GPIO_PIN_RESET);
159     HAL_GPIO_WritePin(GPIOB, ROW4_Pin, GPIO_PIN_SET);
160     break;
161   case 5: // Enable row 5
162     HAL_GPIO_WritePin(GPIOB, ROW4_Pin, GPIO_PIN_RESET);
163     HAL_GPIO_WritePin(GPIOB, ROW5_Pin, GPIO_PIN_SET);
164     break;
165   case 6: // Enable row 6
166     HAL_GPIO_WritePin(GPIOB, ROW5_Pin, GPIO_PIN_RESET);
167     HAL_GPIO_WritePin(GPIOB, ROW6_Pin, GPIO_PIN_SET);
168     break;
169   case 7: // Enable row 7
170     HAL_GPIO_WritePin(GPIOB, ROW6_Pin, GPIO_PIN_RESET);
171     HAL_GPIO_WritePin(GPIOB, ROW7_Pin, GPIO_PIN_SET);
172     break;
173   default:
174     break;
175   }
176   // Update active row data
177   updateLEDMatrix(index_led_matrix);
178   // Move to next row
179   index_led_matrix++;
180   if (index_led_matrix >= MAX_LED_MATRIX)
181   {
182     index_led_matrix = 0; // Back to row 0
183   }
184 }
185
```

```
186 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
187 {
188   if (htim->Instance == TIM2)
189   {
190     // Update all software timers every 10ms
191     timerRun();
192     // --- TIMER 0: Scan 7-seg display ---
193     if (isTimerExpired(0))
194     {
195       setTimer(0, 25); // 250ms
196       scanLED();
197     }
198     // --- TIMER 1: Blink dot (:) ---
199     if (isTimerExpired(1))
200     {
201       setTimer(1, 50); // 750ms
202       HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
203     }
204     // --- TIMER 2: Blink red LED ---
205     if (isTimerExpired(2))
206     {
207       setTimer(2, 50); // 500ms
208       HAL_GPIO_TogglePin(GPIOA, LED_RED_Pin);
209     }
210     // --- TIMER 3: Update clock time ---
211     if (isTimerExpired(3))
212     {
213       second++;
214       if (second >= 60) {
215           second = 0;
216           minute++;
217       }
218       if (minute >= 60) {
219           minute = 0;
220           hour++;
221       }
```

```
222       if (hour >= 24) {
223           hour = 0;
224           }
225       updateClockBuffer();
226       setTimer(3, 100); // 1s
227     }
228   }
229   // --- TIMER 4: Scan LED matrix ---
230   if (isTimerExpired(4))
231   {
232     setTimer(4, 10); // 100ms
233     scanLEDMatrix();
234   }
235 }
```

3.10 Exercise 10

Create an animation on LED matrix, for example, the character is shifted to the left.

[Report Solution]:

Report 1: Briefly describe your solution and present your source code in the report.

The source code and Proteus schematic are provided at STM32 Lab 2: LEDMatrix Animation.

```
/* ===== PART 1: GLOBAL VARIABLES FOR 7-SEGMENT CLOCK ===== */
const int MAX_LED = 4;                    // 4 seven-segment LEDs
int index_led = 0;                        // Current LED index (0-3)
int hour = 15, minute = 8, second = 50; // Initial time: 15:08:50
int led_buffer[4] = {0, 0, 0, 0};         // Display buffer for 4 LEDs

/* ===== PART 2: GLOBAL VARIABLES FOR 8x8 LED MATRIX ===== */
const int MAX_LED_MATRIX = 8; // 8 rows
int index_led_matrix = 0;     // Current row index (0-7)
int shiftVar = 0;             // Shift variable (0-8)

/* Pattern for letter "A" (8x8, with 1 empty column for spacing) */
uint16_t matrix_buffer[8] = {
    0x1800, // Row 0
```

```
    0x2400, // Row 1
    0x4200, // Row 2
    0x4200, // Row 3
    0x7E00, // Row 4
    0x4200, // Row 5
    0x4200, // Row 6
    0x4200  // Row 7
};

/* ===== PART 3: FUNCTIONS FOR LED MATRIX EFFECT ===== */

GPIO_PinState convertToBit(uint16_t hexa, int index)
{
  int actualIndex = (index + shiftVar) % 9;       // Apply horizontal shift
  int bitValue = (actualIndex == 8) ? 0
                : (hexa >> (15 - actualIndex)) & 0x01;
                // ON if bit=1, else OFF
  return (bitValue == 1) ? GPIO_PIN_SET : GPIO_PIN_RESET;
}

/*
 * Update a single row of LED matrix according to matrix_buffer data
 * Each bit corresponds to one column (ENM0..ENM7).
 */
void updateLEDMatrix(int index)
{
  HAL_GPIO_WritePin(GPIOA, ENM0_Pin, convertToBit(matrix_buffer[index], 0));
  HAL_GPIO_WritePin(GPIOA, ENM1_Pin, convertToBit(matrix_buffer[index], 1));
  HAL_GPIO_WritePin(GPIOA, ENM2_Pin, convertToBit(matrix_buffer[index], 2));
  HAL_GPIO_WritePin(GPIOA, ENM3_Pin, convertToBit(matrix_buffer[index], 3));
  HAL_GPIO_WritePin(GPIOA, ENM4_Pin, convertToBit(matrix_buffer[
```

```
    index], 4));
46  HAL_GPIO_WritePin(GPIOA, ENM5_Pin, convertToBit(matrix_buffer[
    index], 5));
47  HAL_GPIO_WritePin(GPIOA, ENM6_Pin, convertToBit(matrix_buffer[
    index], 6));
48  HAL_GPIO_WritePin(GPIOA, ENM7_Pin, convertToBit(matrix_buffer[
    index], 7));
49 }
50
51 /*
52  * Scan the LED matrix row by row (multiplexing).
53 */
54 void scanLEDMatrix()
55 {
56   switch (index_led_matrix)
57   {
58   case 0:
59       HAL_GPIO_WritePin(GPIOB, ROW7_Pin, GPIO_PIN_RESET);
60       HAL_GPIO_WritePin(GPIOB, ROW0_Pin, GPIO_PIN_SET);
61       break;
62   case 1:
63       HAL_GPIO_WritePin(GPIOB, ROW0_Pin, GPIO_PIN_RESET);
64       HAL_GPIO_WritePin(GPIOB, ROW1_Pin, GPIO_PIN_SET);
65       break;
66   case 2:
67       HAL_GPIO_WritePin(GPIOB, ROW1_Pin, GPIO_PIN_RESET);
68       HAL_GPIO_WritePin(GPIOB, ROW2_Pin, GPIO_PIN_SET);
69       break;
70   case 3:
71       HAL_GPIO_WritePin(GPIOB, ROW2_Pin, GPIO_PIN_RESET);
72       HAL_GPIO_WritePin(GPIOB, ROW3_Pin, GPIO_PIN_SET);
73       break;
74   case 4:
75       HAL_GPIO_WritePin(GPIOB, ROW3_Pin, GPIO_PIN_RESET);
76       HAL_GPIO_WritePin(GPIOB, ROW4_Pin, GPIO_PIN_SET);
77       break;
```

```
78    case 5:
79        HAL_GPIO_WritePin(GPIOB, ROW4_Pin, GPIO_PIN_RESET);
80        HAL_GPIO_WritePin(GPIOB, ROW5_Pin, GPIO_PIN_SET);
81        break;
82    case 6:
83        HAL_GPIO_WritePin(GPIOB, ROW5_Pin, GPIO_PIN_RESET);
84        HAL_GPIO_WritePin(GPIOB, ROW6_Pin, GPIO_PIN_SET);
85        break;
86    case 7:
87        HAL_GPIO_WritePin(GPIOB, ROW6_Pin, GPIO_PIN_RESET);
88        HAL_GPIO_WritePin(GPIOB, ROW7_Pin, GPIO_PIN_SET);
89        break;
90    default:
91        break;
92    }
93    // Update LED data for the currently active row
94    updateLEDMatrix(index_led_matrix);
95    // Move to the next row
96    index_led_matrix++;
97    if (index_led_matrix >= MAX_LED_MATRIX)
98    {
99      index_led_matrix = 0;           // Reset to first row
100     shiftVar = (shiftVar + 1) % 9;  // Increment horizontal shift (
     loop 0..8)
101   }
102 }
103
104 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
105 {
106   if (htim->Instance == TIM2)
107   {
108     timerRun();
109     // 7-segment scanning (every 25ms for smooth display)
110     if (isTimerExpired(0))
111     {
112       setTimer(0, 25);
```

```
113       scanLED();
114     }
115     // Toggle DOT LED (every 750ms)
116     if (isTimerExpired(1))
117     {
118       setTimer(1, 75);
119       HAL_GPIO_TogglePin(GPIOA, DOT_Pin);
120     }
121     // Toggle RED LED (every 500ms)
122     if (isTimerExpired(2))
123     {
124       setTimer(2, 50);
125       HAL_GPIO_TogglePin(GPIOA, LED_RED_Pin);
126     }
127     // Update clock (every 1s)
128     if (isTimerExpired(3))
129     {
130       second++;
131       if (second >= 60)
132       {
133         second = 0;
134         minute++;
135       }
136       if (minute >= 60)
137       {
138         minute = 0;
139         hour++;
140       }
141       if (hour >= 24)
142       {
143         hour = 0;
144       }
145       updateClockBuffer();
146       setTimer(3, 100);
147     }
148     // LED Matrix scanning (every 10ms)
```

```
149     if (isTimerExpired(4))
150     {
151       setTimer(4, 10);
152       scanLEDMatrix();
153     }
154   }
155 }
```

Tài liệu

[1]